5-2 数据通路和时序系统

- 1. 数据通路的组成
  - 组合逻辑元件
    - **定义**：输出只取决于当前输入。
    - **时钟关系**：组合电路的定时不受时钟信号直接控制。
    - **输出变化**：所有输入信号到达后，经过一定逻辑门延迟，输出端改变，并保持到输入再次改变。
    - **常见元件**：多路选择器 MUX、加法器 Adder、算术逻辑部件 ALU、译码器 Decoder 等。
  - 多路选择器 MUX
    - **作用**：根据选择信号 Select，从多个输入中选一路作为输出。
    - **典型用途**：在数据通路中选择数据来源，例如 ALU 第二操作数来自寄存器还是立即数。
  - 加法器 Adder
    - **作用**：完成加法运算。
    - **典型用途**：计算 `PC + 4` 、分支目标地址、地址偏移等。
  - ALU
    - **作用**：完成算术和逻辑运算。
    - **输出**：运算结果 Result、Zero 标志、Overflow 标志等。
    - **典型用途**：加法、减法、按位或、比较大小等。
  - 译码器 Decoder
    - **作用**：根据输入编码产生相应输出信号。
    - **典型用途**：指令译码、寄存器选择、控制信号生成。
- 2. 状态元件
  - 基本概念
    - **定义**：具有存储功能的元件。
    - **写入控制**：输入状态在时钟控制下写入电路。
    - **保持输出**：写入后保持输出值不变，直到下一次时钟到达。
    - **读写特点**：输入端由时钟决定何时写入，输出端状态随时可以读出。
  - D 触发器时序约束
    - **建立时间 Setup time**：时钟边沿到来前，输入端 D 必须稳定有效的一段时间。
    - **保持时间 Hold time**：时钟边沿到来后，输入端 D 必须继续保持稳定的一段时间。
    - **Clock-to-Q time**：时钟边沿到来后，输出端 Q 改变为输入端 D 状态所需的延迟时间。
    - **正确工作条件**：输入必须同时满足建立时间和保持时间约束。
  - 常见状态元件
    - **寄存器**：保存 CPU 内部数据。
    - **暂存器/锁存器**：临时保存中间值。
    - **IR**：保存当前指令。
    - **PC**：保存下一条指令地址。
    - **通用寄存器组**：保存操作数和运算结果。
    - **特殊寄存器**：输出端带三态门、带清零、带自增、带移位功能的寄存器等。
- 3. 数据通路与时序控制
  - 早期三级时序系统
    - **三级结构**：机器周期、节拍、脉冲。
    - **机器周期**：一个指令周期可分为取指令、读操作数、执行并写结果等多个基本工作周期。
    - **机器周期长度**：不同机器周期长短可能不同，通常以主存工作周期为基础确定。
    - **节拍**：一个机器周期内的若干步动作，每个动作可在一个节拍内完成。
    - **脉冲**：一个节拍内可设置一个或多个工作脉冲，用于产生操作控制信号并配合操作。
  - 现代计算机时钟信号
    - **变化趋势**：现代计算机中不再采用传统三级时序系统，机器周期概念逐渐淡化。
    - **基本定时信号**：整个数据通路中的定时信号就是时钟。
    - **时钟周期**：一个时钟周期就是一个节拍。
    - **CPU 时钟周期**：机器内部主时钟脉冲的宽度，是 CPU 工作的最小时间单位。
  - 时钟周期确定依据
    - **数据通路结构**：可看作由状态元件和组合逻辑元件交替组成。
    - **基本形式**： `状态元件 -> 组合逻辑 -> 状态元件` 。
    - **周期基准**：时钟周期以相邻状态单元之间组合逻辑电路的最大延迟时间为基准确定。
    - **约束因素**：还需要考虑状态元件的建立时间、保持时间、Clock-to-Q 延迟等。
- 4. 总线式数据通路
  - 单总线数据通路
    - **结构特点**：所有数据传输共享同一条内部总线。
    - **总线使用限制**：同一时刻只允许一个部件向总线输出数据。
    - **性能特点**：一个时钟周期只允许传一个数据。
    - **主要部件**：PC、IR、MAR、MDR、通用寄存器、Y、Z、ALU、指令译码器等。
    - **优点**：结构简单，硬件成本较低。
    - **缺点**：总线冲突限制并行传输，指令执行步骤较多。
  - 三总线数据通路
    - **结构特点**：使用 A、B、C 三条内部总线。
    - **寄存器堆**：通用寄存器位于双口寄存器堆中。
    - **并行读出**：允许两个寄存器内容同时输出到 A 总线和 B 总线。
    - **写回通路**：运算结果可经 C 总线写回。
    - **性能特点**：相比单总线结构，执行指令所需步骤大为减少。
- 5. 单周期数据通路设计目标
  - 目标指令
    - **R 型 5 条**： `add rd, rs, rt` 、 `sub rd, rs, rt` 、 `subu rd, rs, rt` 、 `slt rd, rs, rt` 、 `sltu rd, rs, rt` 。
    - **I 型 5 条**： `ori rt, rs, imm16` 、 `addiu rt, rs, imm16` 、 `lw rt, rs, imm16` 、 `sw rt, rs, imm16` 、 `beq rs, rt, imm16` 。
    - **J 型 1 条**： `j target` 。
  - R 型指令功能
    - **add/sub**：从 `rs` 、 `rt` 取数后相加或相减，若溢出则异常处理，否则结果送 `rd` 。
    - **subu**：从 `rs` 、 `rt` 取数后相减，结果送 `rd` ，不进行溢出判断。
    - **slt**：按带符号整数比较 `R[rs] < R[rt]` ，小于则 `rd = 1` ，否则 `rd = 0` 。
    - **sltu**：按无符号数比较 `R[rs] < R[rt]` ，小于则 `rd = 1` ，否则 `rd = 0` 。
  - I 型指令功能
    - **ori**： `R[rt] <- R[rs] | ZeroExt(imm16)` 。
    - **addiu**： `R[rt] <- R[rs] + SignExt(imm16)` ，不进行溢出判断。
    - **lw**： `Addr <- R[rs] + SignExt(imm16)` ，再 `R[rt] <- M[Addr]` 。
    - **sw**： `Addr <- R[rs] + SignExt(imm16)` ，再 `M[Addr] <- R[rt]` 。
    - **beq**：比较 `R[rs]` 与 `R[rt]` ，若相等，则 `PC <- PC + 4 + SignExt(imm16) x 4` 。
  - J 型指令功能
    - **j target**：形成无条件跳转目标地址。
    - **地址拼接**： `PC<31:2> <- PC<31:28> || target<25:0>` ，其中 `||` 表示拼接。
    - **特点**：直接计算目标地址并更新 PC。
- 6. ALU 设计
  - 设计目标
    - **支持运算**：加法、减法、按位或、有符号比较、无符号比较。
    - **目标指令来源**：服务于 11 条目标指令中涉及的算术逻辑操作。
  - 输出信息
    - **Result**：运算结果。
    - **Zero**：结果是否为 0。
    - **Overflow**：是否发生溢出。
  - 整体思路
    - **核心结构**：一个加法器 + 一套逻辑电路 + 多路选择器。
    - **控制信号**：由 ALU 操作控制信号生成部件产生，决定 ALU 执行加、减、或、比较等操作。
    - **减法实现**：通常可通过加法器配合取反和进位实现。
- 7. 取指令部件
  - 功能
    - **根据 PC 取指令**：用 PC 作为指令存储器地址。
    - **计算下一条指令地址**：顺序执行时得到 `PC + 4` 。
    - **更新 PC**：根据顺序、分支或跳转结果写入新 PC。
  - 组成
    - **PC**：程序计数器。
    - **指令存储器**：根据地址输出指令字。
    - **下地址逻辑**：计算下一条指令地址。
  - 下地址情况
    - **顺序指令**： `PC <- PC + 4` 。
    - **转移执行**： `PC <- 转移目标地址` 。
- 8. 各类指令的数据通路
  - R 型指令数据通路
    - **执行流程**：读取 `rs` 、 `rt` ，ALU 完成运算，结果写回 `rd` 。
    - **主要控制信号**：
      - **RegWr**：是否写寄存器。
      - **ALUCtr**：选择 ALU 运算类型。
    - **写回目标**： `rd` 。
  - 立即数运算指令数据通路
    - **基础**：在 R 型数据通路基础上扩展。
    - **新增 RegDst MUX**：目的寄存器由 `rd` 变为 `rt` 。
    - **新增扩展器 ExtOp**：将 16 位立即数扩展为 32 位。
    - **新增 ALUSrc MUX**：ALU 第二操作数可选择来自寄存器或立即数。
  - Load/Store 数据通路
    - **基础**：在立即数运算数据通路基础上扩展。
    - **新增数据存储器**：支持数据读写。
    - **新增 MemtoReg MUX**：决定写回寄存器的数据来自 ALU 结果还是数据存储器。
    - **MemWr**：控制是否写数据存储器。
    - **地址计算**：ALU 计算 `R[rs] + SignExt(imm16)` 。
  - 分支指令数据通路
    - **基础**：在 Load/Store 数据通路基础上扩展。
    - **新增下地址逻辑**：支持条件分支目标地址计算。
    - **新增 Branch 控制逻辑**：根据比较结果决定是否转移。
    - **比较方式**：ALU 执行减法，利用 Zero 判断 `rs` 与 `rt` 是否相等。
  - 无条件转移指令数据通路
    - **基础**：在支持 Branch 的取指部件上继续扩展。
    - **新增 Jump 目标地址生成逻辑**：支持无条件跳转地址形成。
    - **新增 Jump 控制 MUX**：下一条 PC 可选择 Jump 目标地址。
  - 综合 11 条指令的完整单周期数据通路
    - **取指令部件**：提供指令字和 PC 更新。
    - **寄存器堆**：读 `rs` 、 `rt` ，写 `rd` 或 `rt` 。
    - **扩展器**：处理 `imm16` 的符号扩展或零扩展。
    - **ALU**：完成运算、地址计算、比较。
    - **数据存储器**：支持 `lw` 和 `sw` 。
    - **多路选择器**：选择写回寄存器、ALU 输入、写回数据、下一条 PC。
    - **控制信号**：包括 `RegDst` 、 `RegWr` 、 `ExtOp` 、 `ALUSrc` 、 `ALUctr` 、 `MemtoReg` 、 `MemWr` 、 `Branch` 、 `Jump` 等。